

LES POINTEURS

I. Introduction

Toute variable manipulée dans un programme est stockée quelque part en mémoire centrale. La mémoire peut être assimilée à un tableau dont chaque case possède une adresse unique. Pour accéder au contenu d'une variable, il suffit, donc, de connaître l'adresse de la case mémoire allouée (réservée) pour elle.

Une variable est caractérisée par 3 informations:

- son identifiant : le nom de la variable mentionné dans la section VAR de l'algorithme
- son adresse : l'adresse mémoire ou l'adresse physique où son contenu est stocké
- sa valeur ou son contenu

Exemple

i, j : entier

i ← 1

j ← i

le système d'exploitation a placé la variable i à l'adresse 4831830000 et j à l'adresse 5254136954 →

identifiant	Adresse	contenu
i	4831830000	1
j	5254136954	1

En plus d'accéder au contenu d'une variable à travers son nom (identifiant), il peut être plus intéressant ou tout simplement nécessaire d'y accéder directement par son adresse.

Dans ce cas, nous aurons besoin de disposer d'une variable qu'on va appeler **PP** qui va contenir l'adresse de la variable à laquelle on veut accéder. PP est d'un type particulier appelé **Pointeur**.

II. Définitions

1 Définition

Un pointeur est une variable spéciale qui contient l'adresse d'un emplacement mémoire.

Si un pointeur P contient l'adresse d'une variable A, on dit que 'P pointe sur A'.

2. Déclaration

2.1 déclaration d'une variable de type pointeur

Var

nomvar : Pointeur sur Type_contenu (contenu de l'adresse mémoire stockée dans nomvar).

2.2 déclaration d'un type Pointeur

TYPE

NouvType = Pointeur sur Type_contenu

Var

Nomvar : NouvType

Exemple

Type

PE = pointeur sur entier

Var

P1, P2 : PE

3 Récupérer l'adresse d'une Variable

Pour connaître (récupérer) l'adresse d'une variable nous disposons de l'opérateur unaire & .

Par l'exemple :

i: entier

écrire (" l ' adresse de i = " ,& i)

Si le compilateur a placé la variable i à l'adresse 4831830000 alors l'affichage sera :

l'adresse de i = 4831830000

Exemple 1 :

i : entier

i ← 3

p : Pointeur sur entier

p ← &i ;

identifiant	Adresse	contenu
i	4830000	3
p	1623000	4830000

Exemple 2 :

TYPE

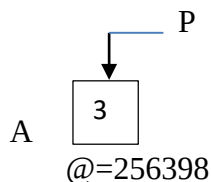
PE = Pointeur sur entier

VAR

P : PE

A : entier

Représentation mémoire



4. Accès au contenu d'un pointeur (*)

A : entier

p : Pointeur sur entier

A ← 3 → Instruction d'affectation de la valeur 3 à la case mémoire correspondant à A

p ← &A → affectation de l'adresse physique de la case mémoire à la variable P

écrire ("La valeur de *p =" , *p) → cette instruction permet d'afficher la valeur 3

*P ← 12 → modification de contenu de la case mémoire pointée par P

Remarque

*P et A permettent de faire référence à la même case mémoire (d'adresse 256398 par exemple)

5. Arithmétique des pointeurs

La valeur d'un pointeur est un entier.

On peut appliquer à un pointeur quelques opérations arithmétiques :

- Addition d'un entier à un pointeur.
- Soustraction d'un entier à pointeur.
- Différence entre deux pointeurs (de même type)

5.1 L'addition

Soit i un entier, p et q deux pointeurs sur un élément de même type.

L'expression $q \leftarrow p + i$ (ou $q \leftarrow p - i$) permet de pointer sur la case mémoire suivante (ou précédente) de la case qui est référencée par p .

Exemple

$p1, p2$: pointeur sur entier

i : entier

$i \leftarrow 5$

$p1 \leftarrow \&i + 2$

$p2 \leftarrow p1 - 2$

$\&i = 4830000$:

identifiant	Adresse	contenu
i	4830012	5
$p1$		4830014
$p2$		4830012

5.2 La différence

Soit p et q deux pointeurs sur des éléments de type TYPE,

Exemple :

i, j : entier

$p1, p2$: pointeur sur entier

$i \leftarrow 5$

$p1 \leftarrow \&i + 2$

$p2 \leftarrow p1 - 2$

$j \leftarrow p1 - p2$

identifiant	Adresse	Contenu
i	4830012	5
$p1$		4830014
$p2$		4830012
j		2

III. Création de variables dynamiques

1. Initialisation d'un pointeur

Avant toute utilisation, un pointeur déclaré doit être initialisé

- soit par l'affectation d'une valeur nulle à un pointeur : $p \leftarrow \text{NULL}$
- soit par l'affectation de l'adresse d'une autre variable : $p \leftarrow \&i$
- soit par l'allocation dynamique d'un espace-mémoire libre.

2. Allocation dynamique

L'allocation dynamique est l'opération qui consiste à réserver un espace-mémoire d'une taille définie.

Instruction :

Var

p : Pointeur sur TYPE

n : TYPE

Début

p ← allouer (taille (TYPE))

n ← valeur

*p ← n

Fin

3. Libération mémoire

C'est l'opération qui consiste à libérer l'espace-mémoire alloué.

Tout espace-mémoire alloué dynamiquement via la primitive Allouer (ou équivalent) doit obligatoirement être désalloué par la primitive Libérer, sinon nous rencontrons le fameux problème de fuite mémoire.

IV. Pointeurs et Tableaux à une dimension

Tout tableau est en fait un pointeur constant. L'identificateur du tableau contient l'adresse de la première case du tableau.

Les autres éléments sont accessibles soit par les indices soit en ajoutant des valeurs entières à l'identificateur du tableau.

Exemple :

Var

T : tableau de 20 entier

i, n : entier

début

écrire ("donnez le nombre de cases : ")

lire (n)

Pour i de 0 à n-1

Ecrire ("donnez un entier : ")

Lire (*(T+i))

Finpour

Fin

IV. Manipulation de tableau alloué dynamiquement

Les tableaux statiques reçoivent de la mémoire au moment de la compilation alors que les tableaux dynamiques se voient allouer de la mémoire au moment de l'exécution.

Les tableaux dynamiques sont des tableaux dont la taille n'est définie que lors de l'exécution des instructions du programme, toutefois il ne sera pas possible d'augmenter cette taille par la suite. il y a différence entre structure dynamique et structure statique : un tableau dynamique n'est pas une structure dynamique, à la différence des listes chaînées, et arbre puisque ça taille réservée reste fixe tout au long de l'exécution du module.

V. Passage des paramètres par variable

Soit l'algorithme suivant :

Algorithme PAR_VALEUR

VAR

X, Y : entier

DEBUT (P.P)

X ← 30

Y ← 40

Écrire ("Avant appel de PERMUTER : X=", X, "Y = " , Y)

PERMUTER (X, Y)

Écrire ("Après appel de PERMUTER : X=", X, "Y = " , Y)

FIN

Procédure PERMUTER (A, B : entier)

VAR

aux : entier

DEBUT

aux ← A

A ← B

B ← aux

Écrire ("Dans PERMUTER : A=" , A , "B= " , B)

FIN

Exécution :

Avant appel de PERMUTER : X = 30 Y = 40

Dans PERMUTER : A = 40 B = 30

Après appel de PERMUTER : X = 30 Y = 40

X et Y restent échangés.

➔ Lors de l'appel, les valeurs de X et Y sont copiées dans les paramètres A et B. PERMUTER échange bien le contenu des variables locales A et B, mais les valeurs de X et Y restent inchangées.

➔ Pour changer la valeur d'une variable à l'appel d'une procédure, il faut que :

- l'algorithme appelant fournit **l'adresse de la variable à modifier**
- La procédure appelée dispose dans son entête du **paramètre comme pointeur**.

L'algorithme et la procédure deviennent comme suit :

Procédure PERMUTER (A, B : pointeur sur entier)

VAR

aux : entier

DEBUT

aux ← *A

*A ← *B

*B ← aux

Écrire ("Dans PERMUTER : A=" , *A , "B= " , *B)

FIN

DEBUT (P.P)

X ← 30

Y ← 40

Écrire ("Avant appel de PERMUTER : X=", X, "Y = " , Y)

PERMUTER (&X, &Y)

Écrire ("Après appel de PERMUTER : X=", X, "Y = " , Y)

FIN

Exécution :

Avant appel de PERMUTER : X = 30 Y = 40

Dans PERMUTER : A = 40 B = 30

Après appel de PERMUTER : X = 40 Y = 30

Lors de l'appel, les adresses de X et Y sont copiées dans les pointeurs A et B.

PERMUTER échange ensuite le contenu des adresses indiquées par les pointeurs A et B.